

해클(Hackle) 이벤트 로그 기반 투표 활동 분석

투표 로그 제한 문제에 대한 교차검증 결과 — 작성일: 2026-07-29

대상 DB: mysql 컨테이너(포트 3307) · final / hackle 스키마

목차

- 1. 배경 — 왜 이 교차검증이 필요했나
- 2. 방법론 및 데이터 정제 노트
- 3. 핵심 발견 1 — 해클 로그인 top-10 외 학교 활동이 대량 존재
- 4. 핵심 발견 2 — "완료" 이벤트 대부분이 DB에 흔적을 남기지 않음
- 5. 핵심 발견 3 — 활동은 소수 학교가 아니라 전국에 분산
- 6. 핵심 발견 4 — "무엇에 투표했는지"는 이 데이터로 추적 불가능
- 7. 해석 및 시사점
- 8. 분석 방향 제안
- 9. 한계 및 유의사항

1. 배경 — 왜 이 교차검증이 필요했나

final DB 기준으로 투표 관련 영구 기록 테이블(polls_questionset , accounts_userquestionrecord)에는 전체 5,951개 학교 중 정확히 **10개 학교**(전체 가입자의 0.75%)의 데이터만 존재하고, 나머지 5,541개 학교는 예외 없이 활동 0건으로 나타난다. 이 수치를 그대로 받아들이면 "10개 학교 외에는 유저들이 투표 기능을 아예 쓰지 않았다"는 결론이 되지만, 이것이 **진짜 활동이 없었던 것인지**, 아니면 **서버가 기록만 하지 않은 것인지**를 구분할 방법이 final DB 단독으로는 없었다.

별도로 적재된 hackle DB(해클 클라이언트 이벤트 트래킹 덤프, 2023-07-18~08-10 24일치)에 질문/투표 관련 클릭 이벤트가 있다는 걸 확인하고, 이걸 final DB와 조인해 실제로 top-10 외 학교에서도 투표 관련 사용자 행동이 있었는지 직접 쿼리로 검증했다.

2. 방법론 및 데이터 정제 노트

재현 시 반드시 감안해야 할 정제 이슈 두 가지가 있었다.

2-1. top-10 학교 목록 재확인

final.polls_questionset 을 accounts_user → accounts_group 으로 조인해 학교별 완료 세트를 다시 집계, 기존 리포트와 동일한 10개 school_id를 확인했다: 1719, 5520, 4426, 5491, 369, 271, 4516, 5372, 1478, 352 .

2-2. hackle_properties의 session_id 팬아웃 문제

`hackle.hackle_properties` 는 세션당 1행이 아니라 **세션당 여러 스냅샷 행**(총 525,350행 / distinct `session_id` 253,616개)이 존재한다. 단순히 `hackle_events` 와 `session_id` 로 조인하면 세션당 최대 9행까지 중복 매칭되어 이벤트 건수가 최대 2배 이상 부풀려진다(직접 확인: 최초 조인 시 `click_question_open` 건수가 원본 816,801건보다 많은 1,481,031건으로 집계됨). → `DISTINCT session_id, user_id` 로 정제한 서브쿼리를 만들어 사용.

2-3. user_id 스키마 혼재 (숫자 ID vs Firebase UID)

같은 세션 안에 **숫자형 user_id**(`final.accounts_user.id` 와 매핑 가능)와 **Firestore 문자열**이 동시에 찍히는 경우가 151,814개 세션(전체의 약 60%)에서 발견됐다. 샘플 확인 결과 이는 동일 인물이 버전 업데이트/로그인 시점에 따라 두 가지 ID 체계로 번갈아 기록된 것으로 보이며(예: 같은 세션에서 `1189864` 와 `BZu1Jkzkd502AfSkIzDXzJgPxbu2` 가 함께 발견), 진짜 계정 충돌(세션 하나에 서로 다른 두 숫자 ID)은 62건 뿐으로 무시 가능한 수준이다. → `user_id REGEXP '^[0-9]+$'` 로 숫자형만 필터링해 `final.accounts_user` 와 안전하게 조인.

이 정제를 거치지 않으면?

정제 전 단순 조인은 이벤트 건수를 최대 2배가량 부풀리고, 학교 매핑 자체도 왜곡할 수 있다. 이후 모든 수치는 위 정제 과정을 거친 뒤의 결과다.

3. 핵심 발견 1 — 해클 로그인 top-10 외 학교 활동이 대량 존재

정제된 `session` → 숫자 `user_id` 매핑을 `hackle_events` 에 연결하고, `final.accounts_user` → `accounts_group` 으로 학교를 붙여 top-10 vs 나머지(OTHER)로 나눠 집계했다.

이벤트	학교 그룹	건수	distinct 유저 수
click_question_open	TOP10	3,034	598
	OTHER	802,390	110,408
click_question_start	TOP10	607	351
	OTHER	217,010	73,880
click_question_ask	TOP10	742	379
	OTHER	133,612	70,285
complete_question	TOP10	359	211
	OTHER	151,903	48,770
skip_question	TOP10	839	159
	OTHER	448,756	38,374

"11위 학교부터 활동 0건"은 final DB의 영구 저장 테이블 기준으로만 사실이다. 클라이언트 이벤트 로그 기준으로는 top-10이 아닌 학교의 유저 4만~11만 명 규모가 질문 열람·시작·완료·스킵 행동을 실제로 하고 있었다.

4. 핵심 발견 2 — "완료" 이벤트 대부분이 DB에 흔적을 남기지 않음

그렇다면 이 `complete_question` 이벤트를 낸 OTHER 학교 유저 49,623명은 실제로 `final` DB에 투표 기록이 남았을까?

지표	값
hackle에서 <code>complete_question</code> 이벤트를 낸 OTHER 학교 유저 수	49,623명
<code>final.polls_questionset</code> 에 조금이라도 기록이 있는 유저	211명 (0.4%)
<code>final.accounts_userquestionrecord</code> 에 조금이라도 기록이 있는 유저	210명 (0.4%)

즉 **49,412명(99.6%)**은 앱에서 "투표 완료"에 해당하는 이벤트가 분명히 발생했음에도, 서버 DB에는 그 어떤 흔적도 없다. 이것이 시간 범위 문제(해클 추적 기간이 짧아서 final DB 관측 구간과 안 겹치는 것)가 아닌지도 확인했다: `final.polls_questionset` / `accounts_userquestionrecord` 는 해클 추적 기간(2023-07-18~08-10) 내에서든, 전체 서비스 기간을 통틀어서든 OTHER 학교 기준 **완전히 0건**이다.

즉 데이터 유실이나 관측 기간 문제가 아니라, 구조적으로 top-10 외 학교의 투표 결과는 서버에 저장되지 않는다.

5. 핵심 발견 3 — 활동은 소수 학교가 아니라 전국에 분산

혹시 11~15위권 등 "컷오프 바로 아래" 소수 학교에 활동이 몰려있는 게 아닌가 확인했으나, 그렇지 않았다.

지표	값
<code>complete_question</code> 이벤트가 발생한 OTHER 학교 수	4,155개교
상위 활동 학교 예시 (공식 학생수)	57명, 75명, 69명, 47명, 30명 등 — 소규모 학교 위주로 넓게 분산

즉 top-10에 들지 못한 **대부분의 학교**에서 소수의 유저들이 산발적으로 투표 기능을 사용하려 시도했다는 뜻이다. "판이 열린 학교가 10곳뿐"이라는 서버 기준 사실과 별개로, 클라이언트 단에서는 사실상 전국 단위로 기능이 노출·시도되고 있었다.

6. 핵심 발견 4 — "무엇에 투표했는지"는 이 데이터로 추적 불가능

투표 완료 규모는 확인되지만, 콘텐츠 레벨(어떤 질문에 투표/완료했는지)까지 해클 데이터로 복원할 수 있는지 이벤트별 `question_id` 필드를 확인했다.

이벤트	question_id 채워짐	item_name / page_name
skip_question	98.8% (449,484 / 454,981)	비어있음 — 하지만 question_id로 충분
complete_question	0% (전부 NULL)	둘 다 공백
click_question_start	0% (전부 NULL)	page_name 전부 "home" 고정값
click_question_open	0% (전부 NULL)	둘 다 공백
click_question_ask	0% (전부 NULL)	page_name 전부 "home" 고정값

`skip_question` 의 `question_id` 는 `final.polls_question` 과 100% 정합(449,484건 전부 매칭)되어, "어떤 질문이 스킵됐는지"는 완벽히 추적 가능하다(예: "화해하고 싶은 친구는?" 403회, "전화를 많이 거는 사람은?" 342회 등).

그러나 완료/열람/시작 이벤트는 애초에 어떤 질문이었는지를 기록하도록 계측되어 있지 않다. 이런 데이터 유실이 아니라 계측 설계 자체의 공백으로 보인다 — 유저·시각·세션 단위 집계는 가능해도, 콘텐츠(질문) 단위 분석은 skip 행동에 대해서만 가능하다.

7. 해석 및 시사점

- 클라이언트 UI/투표 플로우는 사실상 모든 학교에 열려 있고, 유저들은 실제로 사용을 시도한다. open → start → complete 퍼널이 top-10이 아닌 학교에서도 정상적인 비율로 흘러간다(예: OTHER 그룹 click_question_open 802,390건 대비 complete_question 151,903건).
- "투표 로그가 제한적"인 근본 원인은 유저 행동 부족이 아니라, 서버 측에서 top-10 외 학교의 결과를 저장하지 않거나 조용히 누락시키는 로직/게이팅에 있을 가능성이 높다. API 응답 코드나 서버 에러 로그를 확인하기 전까지는 "실패해서 안 쌓인 것"과 "성공했지만 기록만 안 되는 것"을 구분할 수 없다.
- 따라서 분석 설계 시 "활동이 없었다"와 "기록되지 않았다"를 명확히 구분해서 다뤄야 한다 — 지금까지의 리포트(예: 핵심 발견 1 "10개 학교에서만 돌아간다")는 서버 기록 기준으로는 정확하지만, 유저 관심/수요 관점에서는 과소평가된 수치일 수 있다.
- 해클 데이터는 "완료를 시도한 규모"는 보완해주지만, "무엇에 투표했는지" 콘텐츠 레벨 분석에는 쓸 수 없다 (skip 행동 제외).

8. 분석 방향 제안

- 규모/퍼널 분석은 top-10 학교로 한정하지 않고 해클 이벤트(open/start/complete/skip) 기준으로 전사 확장 가능 — 단, "서버에 저장된 진짜 투표"가 아니라 "시도 규모"라는 점을 명시.

2. **콘텐츠(질문) 레벨 분석**은 현재로선 `final` DB가 있는 top-10 학교 데이터로만 가능 — 결과 제시 시 "이 결과는 서비스 전체가 아니라 투표 기능이 활성화된 10개 학교 한정"이라는 caveat을 반드시 명시.
3. **근본 원인 확인이 선행되어야 함**: 백엔드 코드/서버 로그에서 top-10 외 학교의 투표 완료 요청이 (a) 애초에 거부되는지 (b) 성공 응답은 주지만 저장 로직에서 걸러지는지 확인 필요 — 이에 따라 "유실된 잠재 데이터를 복구할 수 있는지" 여부가 갈림.
4. **skip_question**은 유일하게 **question_id**가 보존된 이벤트이므로, "어떤 질문이 스킵되는가"는 top-10 외 학교까지 포함해 전사 단위로 유일하게 콘텐츠 레벨 분석이 가능한 지표다. 활용 가치가 있음.

9. 한계 및 유의사항

이 리포트의 수치를 인용할 때 함께 고려해야 할 점

- 핵클 추적 기간은 2023-07-18~08-10, 단 24일. 서비스 전체 기간(14개월) 대비 매우 좁은 창이며, 이 리포트의 모든 수치는 이 24일 스냅샷 기준이다. 다른 기간에도 동일한 패턴인지는 확인되지 않았다.
- `user_id` 매핑 시 숫자형만 사용했기 때문에, Firebase UID로만 남아있는 유저(전체 `hackle_properties`의 약 36%)는 이번 분석에서 제외됐다 — 실제 규모는 여기 제시된 수치보다 더 클 가능성이 있다.
- `complete_question` 등 이벤트가 정말 "서버에 완료 요청을 전송함"을 의미하는지, 혹은 단순 클라이언트 측 화면 전환/애니메이션 트리거인지는 클라이언트 코드 확인 없이 100% 확정하기 어렵다 — 다만 앞뒤 이벤트 시퀀스(open→start→complete)가 자연스러운 사용자 플로우를 이루고 있어 실제 사용자 행동으로 보는 것이 합리적이다.